



南開大學
Nankai University

计算机学院
并行程序设计期末报告

基于 HIP 的 NTT 并行化与模乘优化

姓名：强徽东

学号：2411764

专业：计算机科学与技术

2026 年 6 月 20 日

目录

1 实验目的	2
2 实验环境	2
3 算法原理	2
3.1 数论变换	2
3.2 GPU 并行映射	3
3.3 旋转因子存储	3
3.4 Barrett 规约	3
3.5 Montgomery 规约	3
3.6 LDS 分块	4
4 程序实现与正确性验证	4
4.1 三部分优化的核心代码	4
4.1.1 优化一：预处理旋转因子	4
4.1.2 优化二：蝶形运算并行化	5
4.1.3 优化三：Barrett 与 Montgomery 模乘	6
5 实验结果与分析	8
5.1 Workgroup 大小	8
5.2 完整 NTT 的模乘与 LDS 优化	9
5.3 独立模乘性能	9
5.4 不同 NTT 规模	10
5.5 rocprofv3 分析	11
6 误差来源与实验局限	11
7 结论	11

1 实验目的

学习并掌握 GPU 编程，深入了解相关内容并对 NTT 算法进行 GPU 优化，以及进行性能测试。

2 实验环境

表 1: 软硬件环境

项目	配置
CPU	AMD Ryzen AI MAX+ PRO 395
GPU	AMD Radeon 8060S Graphics (gfx1151)
GPU 架构	RDNA 3.5
计算单元	40 CU (HIP 属性接口可按 20 WGP 报告)
Wavefront	32
每个 CU 的 SIMD 数	2
每个 workgroup 的最大线程数	1024
每个 CU 的 LDS	64 KB
HIP 版本	7.13.99004-3309c6114a
编译器	AMD Clang 23.0.0git
编译参数	hipcc -O3 -std=c++17
性能分析工具	rocpv3

3 算法原理

3.1 数论变换

NTT 是在有限域上执行的离散傅里叶变换。本实验选用

$$q = 998244353 = 119 \times 2^{23} + 1, \quad g = 3,$$

其中 g 是模 q 的原根。由于 $q - 1$ 含有因子 2^{23} ，本实验可处理的 radix-2 NTT 最大长度为 2^{23} 。

对于长度为 N 的变换， N 次单位根为

$$\omega_N = g^{(q-1)/N} \bmod q.$$

每个蝶形运算为

$$u = a_i, \tag{1}$$

$$v = a_{i+L/2} \omega_L^j \bmod q, \tag{2}$$

$$a_i = (u + v) \bmod q, \tag{3}$$

$$a_{i+L/2} = (u - v + q) \bmod q, \tag{4}$$

其中 L 为当前阶段的蝶形长度。逆变换使用逆单位根，并在最后乘以 $N^{-1} \bmod q$ 。

3.2 GPU 并行映射

首先在 GPU 上执行位逆序置换。随后每个线程负责一个蝶形。设线程全局编号为 t ，则线程对应的数据位置为

$$h = L/2, \quad (5)$$

$$\text{group} = \lfloor t/h \rfloor, \quad (6)$$

$$j = t \bmod h, \quad (7)$$

$$i_0 = \text{group} \cdot L + j, \quad (8)$$

$$i_1 = i_0 + h. \quad (9)$$

同一阶段的蝶形彼此独立，因此可以并行执行。不同 workgroup 之间不存在可由块内同步函数实现的全局屏障，所以普通版本每个 NTT 阶段启动一次 kernel。对于 $N = 2^{20}$ ，一次正变换包含 20 次蝶形 kernel。

3.3 旋转因子存储

CPU 预先生成正变换和逆变换旋转因子表，然后通过 `hipMemcpy` 一次性复制到 GPU 全局内存。第 L 阶段第 j 个蝶形使用的表下标为

$$j \frac{N}{L}.$$

这种方法避免了每个线程在 kernel 中执行模幂运算。旋转因子的生成时间和首次主机到设备传输时间不计入纯 kernel 时间。

3.4 Barrett 规约

预计算

$$\mu = \left\lfloor \frac{2^{64}}{q} \right\rfloor.$$

对于 $x = ab$ ，利用 64 位乘法高位近似商：

$$\hat{m} = \left\lfloor \frac{x\mu}{2^{64}} \right\rfloor, \quad (10)$$

$$r = x - \hat{m}q. \quad (11)$$

最后通过条件减法将 r 调整到 $[0, q)$ 。GPU 使用高位乘法指令 `__umul64hi`，避免运行时整数除法。

3.5 Montgomery 规约

令 $R = 2^{32}$ ，并预计算

$$q' = -q^{-1} \bmod R.$$

对于 $t = ab$ ，Montgomery 规约为

$$m = (t \bmod R)q' \bmod R, \quad (12)$$

$$u = \frac{t + mq}{R}. \quad (13)$$

若 $u \geq q$ ，再减去一次 q 。输入数据、旋转因子和 N^{-1} 都提前转换为 Montgomery 形式 $xR \bmod q$ ，整个 NTT 始终在 Montgomery 域中计算，最后统一转换回普通表示。这样可以避免在每次模乘前后进行域转换。

3.6 LDS 分块

根据 workgroup 测试结果，分块版本采用 128 个线程和 256 个元素的 tile。每个线程负责两个元素的加载或一组蝶形，将前 8 个阶段 ($L = 2, 4, \dots, 256$) 融合到一个 kernel，并在 LDS 中完成。阶段之间使用 `__syncthreads()`。从 $L = 512$ 开始，蝶形会跨越 tile，仍需使用全局内存和独立 kernel。

4 程序实现与正确性验证

程序包含以下版本：

1. CPU 串行 NTT;
2. GPU 运行时 % mod baseline;
3. GPU Barrett 规约版本;
4. GPU Montgomery 规约版本;
5. GPU Montgomery + LDS tiled 版本。

4.1 三部分优化的核心代码

4.1.1 优化一：预处理旋转因子

旋转因子只与模数、变换长度和变换方向有关。程序在 CPU 上一次性生成正、逆变换旋转因子表，随后复制到 GPU 全局内存。kernel 通过下标直接查表，从而避免每个线程在每个阶段重复执行模幂运算。

Listing 1: 旋转因子预计算与传输

```

1 std::vector<u32> make_twiddles(int n, bool inverse) {
2     std::vector<u32> twiddles(n / 2);
3     u32 omega = mod_pow(ROOT, (MOD - 1) / n);
4
5     if (inverse) {
6         omega = mod_pow(omega, MOD - 2);
7     }
8
9     twiddles[0] = 1;
10    for (int i = 1; i < n / 2; ++i) {
11        twiddles[i] = static_cast<u32>(
12            static_cast<u64>(twiddles[i - 1])
13            * omega % MOD
14        );
15    }
16    return twiddles;

```

```

17 }
18
19 std::vector<u32> forward_twiddles =
20     make_twiddles(N, false);
21 std::vector<u32> inverse_twiddles =
22     make_twiddles(N, true);
23
24 hipMalloc(
25     &device_forward_twiddles,
26     forward_twiddles.size() * sizeof(u32)
27 );
28 hipMemcpy(
29     device_forward_twiddles,
30     forward_twiddles.data(),
31     forward_twiddles.size() * sizeof(u32),
32     hipMemcpyHostToDevice
33 );

```

对于长度为 `length` 的阶段，线程使用 $j * (n / \text{length})$ 作为旋转因子下标。正、逆变换分别保存一张表，避免在 GPU 上求逆。预处理和首次传输发生在计时区间之外，正式性能表测量 steady-state NTT kernel 时间。

4.1.2 优化二：蝶形运算并行化

长度为 N 的每个阶段包含 $N/2$ 个互不冲突的蝶形。程序将一个蝶形映射给一个 GPU 线程，使用一维 grid 覆盖所有蝶形。线程先计算所在蝶形组和组内偏移，再完成两次读、一次模乘、模加和模减。

Listing 2: 一个线程负责一个蝶形运算

```

1  __global__ void butterfly_kernel(
2      u32* data,
3      const u32* twiddles,
4      int n,
5      int length,
6      u32 mod
7  ) {
8      int tid = blockIdx.x * blockDim.x + threadIdx.x;
9      if (tid >= n / 2) return;
10
11      int half = length / 2;
12      int group = tid / half;
13      int j = tid % half;
14      int first = group * length + j;
15      int second = first + half;
16      int twiddle_index = j * (n / length);
17
18      u32 left = data[first];
19      u32 right = static_cast<u32>(

```

```

20     static_cast<u64>(data[second])
21     * twiddles[twiddle_index] % mod
22 );
23
24 u32 sum = left + right;
25 if (sum >= mod) sum -= mod;
26
27 u32 difference =
28     left >= right ? left - right
29                   : left + mod - right;
30
31 data[first] = sum;
32 data[second] = difference;
33 }

```

不同阶段存在数据依赖，因此普通版本每个阶段启动一次 kernel，以 kernel 边界完成全局同步。workgroup 大小通过实测选择，而不是直接使用硬件允许的最大值。

Listing 3: 逐阶段启动并行蝶形 kernel

```

1  int butterflies = n / 2;
2  int grid = (butterflies + BLOCK_SIZE - 1)
3           / BLOCK_SIZE;
4
5  for (int length = 2; length <= n; length <=< 1) {
6      hipLaunchKernelGGL(
7          butterfly_kernel,
8          dim3(grid), dim3(BLOCK_SIZE), 0, 0,
9          device_data, device_twiddles,
10         n, length, MOD
11     );
12 }

```

实验还实现了 LDS tiled 版本:以 128 个线程处理 256 个元素,将前 8 个阶段融合到一个 workgroup 内。该版本验证正确,但同步和 LDS 搬运开销使其在本设备上慢于普通并行版本。

4.1.3 优化三: Barrett 与 Montgomery 模乘

Barrett 规约预计算 $\mu = \lfloor 2^{64}/q \rfloor$, 使用乘法高位近似商, 避免运行时整数除法。代码如下。

Listing 4: Barrett 模乘

```

1  constexpr u64 BARRETT_MU =
2      18446744073709551615ULL / MOD;
3
4  __device__ __forceinline__
5  u32 barrett_mul(u32 left, u32 right) {
6      u64 product = static_cast<u64>(left) * right;
7
8      u64 quotient = __umul64hi(

```

```

9         static_cast<unsigned long long>(product),
10        static_cast<unsigned long long>(BARRETT_MU)
11    );
12
13    u64 remainder =
14        product - quotient * static_cast<u64>(MOD);
15
16    if (remainder >= MOD) remainder -= MOD;
17    return static_cast<u32>(remainder);
18 }

```

Montgomery 规约使用 $R = 2^{32}$ ，预计算 $-q^{-1} \bmod R$ ，通过低 32 位乘法和右移完成规约。

Listing 5: Montgomery 规约与模乘

```

1 constexpr u32 inverse_mod_2_32(u32 value) {
2     u32 inverse = 1;
3     for (int i = 0; i < 5; ++i) {
4         inverse *= 2U - value * inverse;
5     }
6     return inverse;
7 }
8
9 constexpr u32 MONT_NINV =
10     0U - inverse_mod_2_32(MOD);
11 constexpr u32 MONT_R_MOD =
12     static_cast<u32>((1ULL << 32) % MOD);
13
14 __host__ __device__ __forceinline__
15 u32 montgomery_reduce(u64 value) {
16     u32 m = static_cast<u32>(value) * MONT_NINV;
17     u64 reduced =
18         (value + static_cast<u64>(m) * MOD) >> 32;
19
20     if (reduced >= MOD) reduced -= MOD;
21     return static_cast<u32>(reduced);
22 }
23
24 __host__ __device__ __forceinline__
25 u32 montgomery_mul(u32 left, u32 right) {
26     return montgomery_reduce(
27         static_cast<u64>(left) * right
28     );
29 }

```

为避免域转换抵消优化收益，输入数据和旋转因子在 NTT 开始前统一转换为 $xR \bmod q$ ，整个正、逆变换均保留在 Montgomery 域中，最终仅转换一次。

Listing 6: Montgomery 域的批量预转换与最终转换


```

1 u32 to_montgomery(u32 value) {
2     return static_cast<u32>(
3         static_cast<u64>(value) * MONT_R_MOD % MOD
4     );
5 }
6
7 for (int i = 0; i < N; ++i) {
8     montgomery_input[i] = to_montgomery(input[i]);
9 }
10 for (u32& value : forward_twiddles) {
11     value = to_montgomery(value);
12 }
13
14 // Convert only after the complete NTT has finished.
15 for (u32& value : gpu_result) {
16     value = montgomery_reduce(value);
17 }

```

在蝶形 kernel 中，只需将普通模乘替换为对应的 Barrett 或 Montgomery 设备函数。线程映射和数据布局保持不变，因此能够单独比较模乘策略的影响。

所有 HIP 调用均检查返回值，并在 kernel 启动后检查 `hipGetLastError()`。GPU 时间使用 HIP Event 测量。测试采用固定随机种子，保证不同版本处理相同输入。

CPU 小规模测试的结果为

Forward NTT:

36 894301004 346334868 201631260 998244349 796613085 651909477 103943341

Inverse NTT:

1 2 3 4 5 6 7 8

Verification: PASSED

在 $N = 2^{20}$ 时，普通取模、Barrett、Montgomery 和 LDS tiled 版本均满足：

- GPU 正变换结果与 CPU 逐元素一致；
- GPU 正变换后执行逆变换能够恢复原始输入；
- 独立模乘测试中三种 GPU 实现均与 CPU 参考结果一致。

5 实验结果与分析

5.1 Workgroup 大小

固定 $N = 2^{20}$ ，使用 Montgomery 版本测试不同 workgroup 大小，每种配置运行 10 次。

表 2: Workgroup 大小对正变换时间的影响

Workgroup 大小	平均时间/ms	标准差/ms
64	1.2249	0.0578
128	1.1147	0.0231
256	1.1331	0.0317
512	1.1773	0.0457
1024	1.1988	0.0359

128 线程取得最佳结果。较小 workgroup 的并行度不足，而较大 workgroup 会降低调度灵活性和活跃 workgroup 数量。由于设备使用 wave32，所有测试值均为 wavefront 大小的整数倍。

5.2 完整 NTT 的模乘与 LDS 优化

固定 $N = 2^{20}$ ，预热 2 次后交错运行各版本 10 次，结果如下。

表 3: 完整 NTT 的不同 GPU 实现

方法	正变换/ms	标准差	逆变换/ms	标准差	正变换相对 baseline
Runtime modulo	1.1523	0.0295	0.7766	0.0022	1.000
Barrett	1.1348	0.0416	0.7522	0.0024	1.015
Montgomery	1.1506	0.0445	0.7504	0.0025	1.002
Montgomery tiled	1.2047	0.0555	0.7791	0.0037	0.957

完整正变换中 Barrett 和 Montgomery 的提升较小。原因是完整 NTT 不仅包含模乘，还包含位逆序、旋转因子读取、全局内存读写和 20 次阶段 kernel 启动。模乘只占总时间的一部分，因此算术优化被其他开销稀释。

LDS tiled 版本比 baseline 慢约 4.3%。虽然它把前 8 个阶段融合为一次 kernel，但增加了 LDS 搬运和多次 workgroup 屏障。对于当前设备和问题规模，这些开销超过了减少 kernel 启动次数带来的收益。因此实验保留该结果，不将分块本身等同于必然加速。

5.3 独立模乘性能

为了隔离模乘开销，使用 $N = 2^{20}$ 个元素，每个线程连续执行 32 次模乘。GPU 预热 2 次并测量 10 次。

表 4: 独立模乘性能

方法	平均时间/ms	标准差/ms	相对 CPU	相对 GPU %
CPU baseline	41.9277	—	1.000	—
Runtime modulo	0.4185	0.0006	100.181	1.000
Barrett	0.1941	0.0095	215.990	2.156
Montgomery	0.0782	0.0022	535.983	5.350

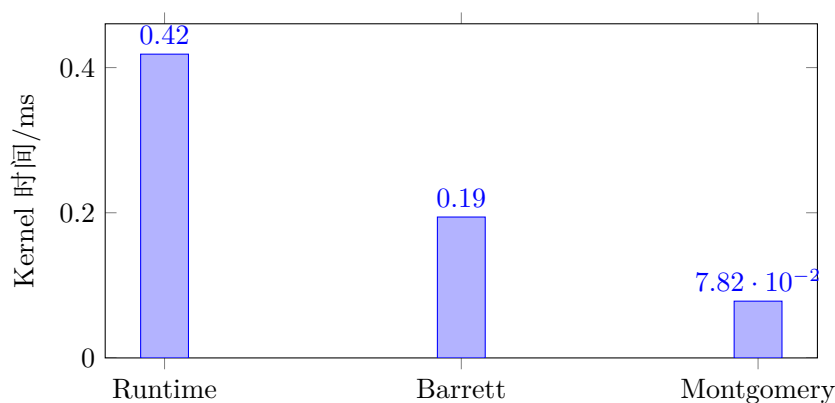


图 5.1: 三种 GPU 模乘方法的独立 kernel 时间

Barrett 和 Montgomery 相对运行时取模分别加速 2.156 倍和 5.350 倍, 证明两种规约可以有效消除 GPU 上昂贵的整数除法。Montgomery 测试的纯 kernel 时间不包含输入和输出的域转换; 这是 steady-state NTT 的合理口径, 因为数据和旋转因子在整个变换期间始终保留在 Montgomery 域中。

5.4 不同 NTT 规模

表 5 给出了不同规模下 CPU 与 GPU 正变换的时间。每个规模运行 5 次并取平均值。

表 5: 不同规模的 NTT 性能

N	CPU/ms	Runtime/ms	Barrett/ms	Montgomery/ms	CPU/Montgomery
1,024	0.018	0.4968	0.4568	0.5222	0.04
16,384	0.374	0.5737	0.5914	0.5452	0.69
262,144	9.064	0.7014	0.6269	0.5815	15.59
1,048,576	31.053	1.1446	1.1539	1.1648	26.66
4,194,304	124.380	8.0227	7.9591	7.9520	15.64
8,388,608	271.823	23.8862	23.6676	23.6723	11.48

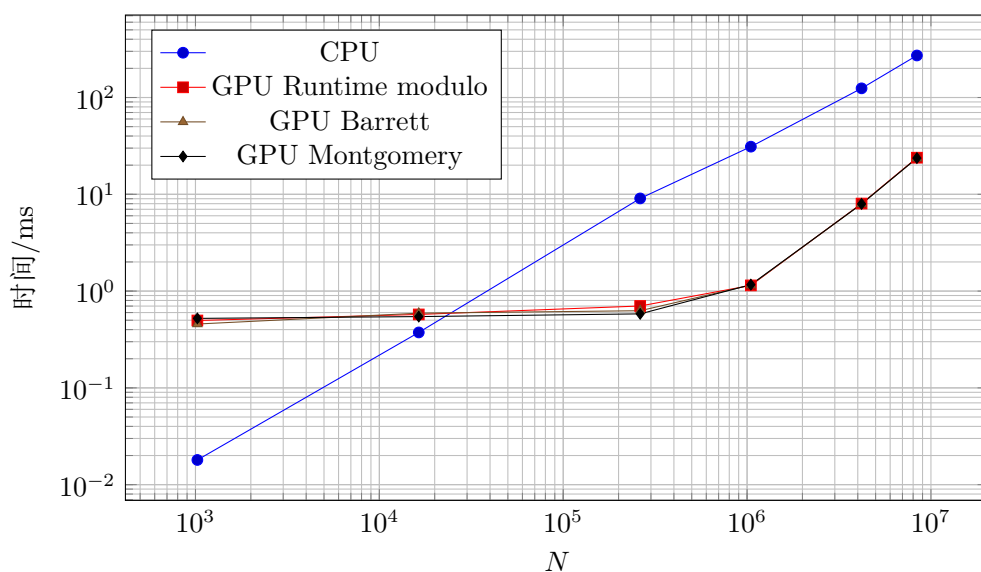


图 5.2: CPU 与 GPU NTT 的规模扩展性

当 $N \leq 2^{14}$ 时, GPU kernel 启动开销占主导, CPU 更快。从 $N = 2^{18}$ 开始, GPU 获得明显优势; $N = 2^{20}$ 时取得本组测试中的最大加速比 26.66。当规模继续增大时, 工作集超出较高级缓存, 位逆序和跨步访存的代价增加, 加速比下降到 15.64 和 11.48。

5.5 rocprofv3 分析

对 $N = 2^{20}$ 的 Montgomery 正、逆变换执行 kernel trace, 结果如下。

表 6: rocprofv3 kernel 统计

Kernel	调用次数	总时间/ns	平均时间/ns	占比
butterfly_kernel	40	1,148,473	28,711.8	78.90%
bit_reverse_kernel	2	296,116	148,058.0	20.34%
scale_kernel	1	10,981	10,981.0	0.75%
合计	43	1,455,570	—	100%

40 次蝶形调用分别来自正变换和逆变换的 20 个阶段。蝶形 kernel 占总 kernel 时间的 78.90%, 是主要热点; 位逆序虽然只调用两次, 但占 20.34%, 也是后续优化的重要方向。性能分析工具本身会引入额外开销, 因此该表用于分析占比, 正式性能表使用未开启 profiler 时的 HIP Event 数据。

6 误差来源与实验局限

1. GPU 表格主要报告纯 kernel 时间, 不包含首次 H2D/D2H 传输、旋转因子生成、设备内存分配和 HIP 上下文初始化;
2. 独立 Montgomery 模乘不包含进入和退出 Montgomery 域的时间, 反映的是 NTT steady-state 场景;
3. CPU baseline 为单线程实现, 未使用 SIMD 或多线程;
4. 本实验固定模数为 998244353, 较为单一;
5. 完整 NTT 的模乘差异较小, 部分差异与标准差处于同一数量级, 因此不应仅凭单次测试宣称显著加速。

7 结论

本实验完成了 CPU 和 HIP GPU 上的 NTT, 并实现了旋转因子 GPU 查表、Barrett 规约、Montgomery 规约和 LDS 分块。所有 GPU 版本均通过 CPU 对照和正逆变换恢复测试。

在独立模乘场景中, Barrett 和 Montgomery 相对 GPU 运行时取模分别取得 2.156 倍和 5.350 倍加速, 说明消除整数除法十分有效。在完整 NTT 中, 访存、位逆序和 kernel 启动占据较大比例, 模乘优化带来的整体提升有限。GPU 对中大规模 NTT 仍表现出明显优势, 在 $N = 2^{20}$ 时相对单线程 CPU 达到 26.66 倍加速。

LDS tiled 版本在当前实现中下降约 4.3%, 表明共享内存优化必须综合考虑同步、数据搬运和 occupancy, 而不能只考虑减少全局内存访问或 kernel 次数。rocprofv3 进一步表明蝶形计算和位逆序

分别占约 78.90% 和 20.34%，后续可以考虑使用 Stockham NTT 消除独立位逆序、使用 wave shuffle 融合小规模阶段，以及针对不同 N 自动选择 tile 和 workgroup 配置。